DAA Assignment 3 - Graphs: DSU, MST, SCC and Shortest Paths

Temirlan Zhangirkhan, group SE-2521.

All numbers come fromresults/results.csv(seed 42, one command reproduces them).

The benchmark fails if Kruskaland Prim disagree, a topologicalorder is invalid, value differs; all 10 reference values are reproduced exactly. Counters are incremented inside the algorithms:

dsuOps = find calls + successful unions;

edgeScans = edges looked at;

comparisons = weight comparisons (sort / heap / relaxation);

maxDepth = deepest explicit DFS stack.

1. Complexity table

Algorithm	Time	Extra space	Justification
DSU find / union	O(alpha(n)) amortized	O(n)	parent[] + rank[]; union by rank keeps height O(log n), path compression flattens it; together inverse Ackermann.
Kruskal	O(E log E)	O(E + V)	Sorting E edges dominates; loop does 2 finds per edge + at most V-1 unions = O(E alpha(V)). Sorted copy O(E), DSU O(V).
Prim (binary heap)	O(E log V)	O(E + V)	Each adjacency entry scanned once (2E) and pushed at most once, each heap op O(log E) = O(log V); lazy heap holds up to O(E) entries.
Kosaraju	O(V + E)	O(V + E)	Two DFS passes, each edge scanned once per pass (2E); reversed graph O(V+E), stack/order/comp O(V).
Kahn	O(V + E)	O(V)	Each vertex enters the queue once, each edge decrements one in-degree once.
DAG shortest paths	O(V + E)	O(V)	One pass: each edge relaxed once in topological order; dist[], parent[], order O(V).

2. Counters vs formulas

Counter divided by the formula must stay constant as n grows (sparse m = 4n, dense m = 20n; k = thousand).

algorithm	counter / formula	density	n=1k	n=3.2k	n=10k	n=31.6k	n=100k	spread
Kruskal	dsuOps / E	sparse	1.791	2.019	2.182	2.232	2.248	25.6 %
Kruskal	dsuOps / E	dense	0.480	0.506	0.465	0.654	0.794	70.8 %
Kruskal	dsuOps / edgeScans (examined edges)	sparse	2.324	2.283	2.259	2.252	2.250	3.3 %
Kruskal	dsuOps / edgeScans (examined edges)	dense	2.232	2.219	2.241	2.166	2.134	5.0 %
Kruskal	comparisons / (E log2 E)	sparse	0.891	0.907	0.869	0.809	0.745	21.8 %
Kruskal	comparisons / (E log2 E)	dense	0.903	0.846	0.784	0.720	0.666	35.5 %
Prim	comparisons / (E log2 V)	sparse	2.196	2.186	2.180	2.178	2.167	1.3 %
Prim	comparisons / (E log2 V)	dense	2.616	2.533	2.473	2.421	2.381	9.9 %
Kosaraju	edgeScans / (V + E)	sparse	1.600	1.600	1.600	1.600	1.600	0.0 %
Kosaraju	edgeScans / (V + E)	dense	1.905	1.905	1.905	1.905	1.905	0.0 %
Kahn	edgeScans / (V + E)	sparse	0.800	0.800	0.800	0.800	0.800	0.0 %
Kahn	edgeScans / (V + E)	dense	0.952	0.952	0.952	0.952	0.952	0.0 %
DagShortestPath	edgeScans / (V + E)	sparse	0.800	0.800	0.800	0.800	0.800	0.0 %
DagShortestPath	edgeScans / (V + E)	dense	0.952	0.952	0.952	0.952	0.952	0.0 %

Kosaraju, Kahn, DAG pass: edgeScans/(V+E) is exactly constant (0.0 % spread): 2E/(V+E) = 1.600 / 1.905 for Kosaraju, E/(V+E) = 0.800 / 0.952 for Kahn and the DAG pass (sparse / dense).

This confirms O(V+E). Prim: comparisons/(E log2 V) = 2.17-2.20 (sparse) and 2.38-2.62 (dense), bounded; the lazy heap is smaller than E, so the real log factor is slightly below log V.

Kruskal: dsuOps/E is not flat because Kruskal stops after V-1 tree edges, so the share of examined edges changes with n (plot 3).

Per examined edge it is flat: dsuOps/edgeScans = 2.13-2.32 (two finds + V-1 unions, i.e. O(E alpha)). Sort comparisons are 0.67-0.91 of E log2 E, bounded by a constant.

3. Slope check (log-log, fit over the five sizes)

edgeScans, dense

1.119

1.000

1.000

1.000

1.000

algorithm	edgeScans, sparse		all operations, sparse	all operations, dense
Kruskal	1.055		1.054	1.022
Prim	1.000		1.101	1.085
Kosaraju	1.000		1.000	1.000
Kahn	1.000		1.000	1.000
DagShortestPath	1.000		1.000	1.000

Traversals (Prim, Kosaraju, Kahn, DAG pass) give slope 1.000. Kruskal's edgeScans are steeper: 1.055 (sparse) and 1.119 (dense), because the number of edges needed until the graph is connected grows like (n/2) ln n, not like n; the sort adds its own E log E.

4. Correctness arguments

Cut property. Let A be part of some MST T, let a cut respect A (no edge of A crosses it), and let e be a lightest edge crossing the cut. If e is in T, we are done.

Otherwise T + e has one cycle; it crosses the cut at least twice, so it contains another crossing edge f of T, and f is not in A.

T' = T - f + e is a spanning tree with w(T') = w(T) - w(f) + w(e) <= w(T), so T' is also minimal and contains A + e; ties are harmless because the inequality is non-strict.

Kruskal: when it takes e = (u,v) with find(u) != find(v), take the cut around the component of u; A does not cross it, and every lighter edge was already examined and either taken or rejected inside one component, so a lighter crossing edge cannot exist; e is safe. Prim: the cut is (tree, rest), A respects it, and the priority queue returns the lightest edge leaving the tree (stale entries to tree vertices are skipped); every step adds a safe edge. Both add V-1 safe edges, hence equal total weight.

Induction on the topological order. Claim: when the pass reaches v, dist[v] equals the true shortest distance d(v) from the super-source (all in-degree-0 vertices have distance 0). Base: the first vertex has in-degree 0, so dist = 0 = d. Step: every path to v ends with an edge (u,v) where u is earlier in the order, so d(v) = min over (u,v) of d(u) + w(u,v). By induction dist[u] = d(u) when u is processed, and u relaxes (u,v) before v is reached, so dist[v] <= d(v). dist[v] is always the length of a real path, so dist[v] >= d(v). Thus dist[v] = d(v) and never changes again; one pass over all edges, O(V+E), is enough.

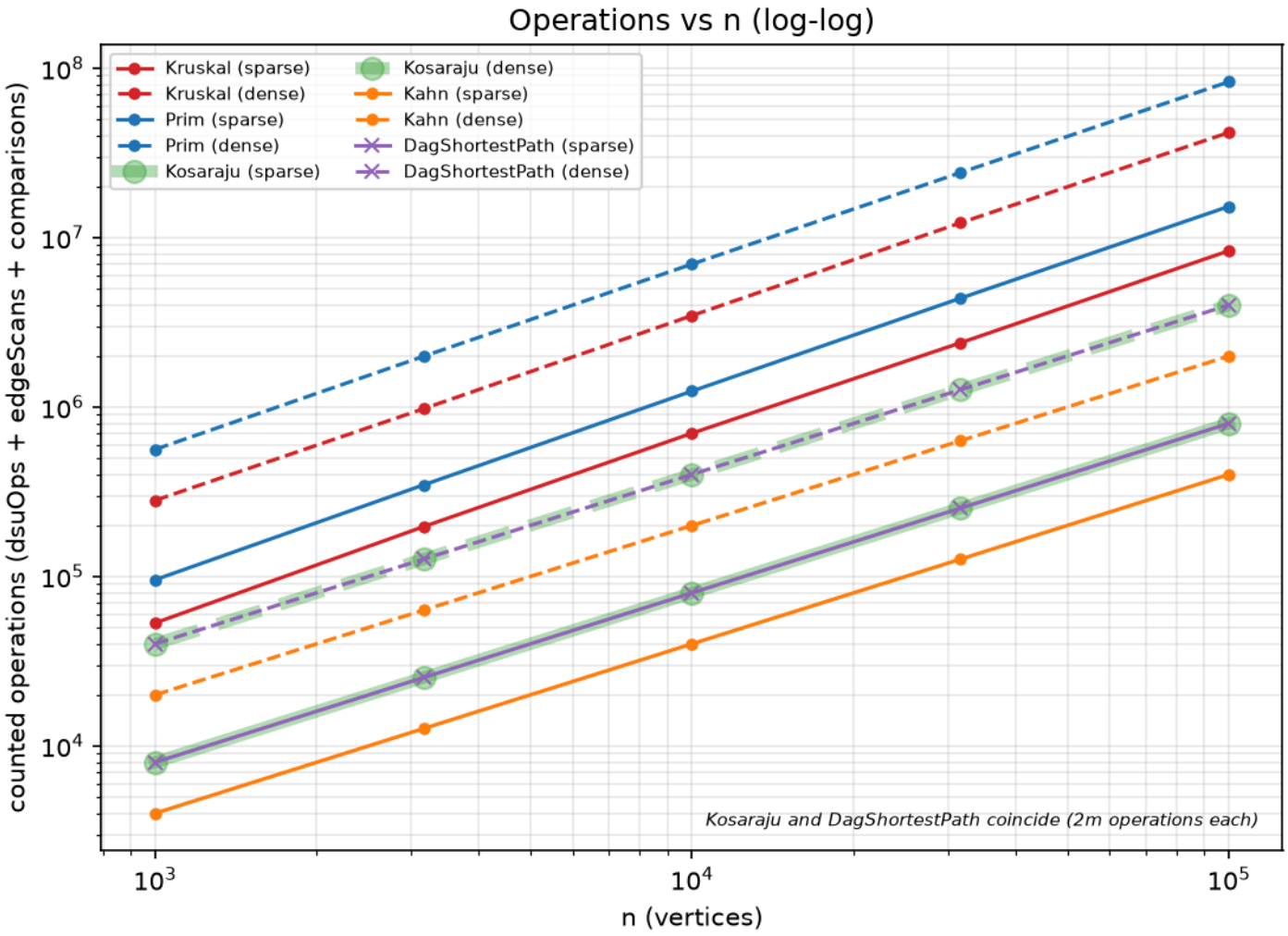
Negative weights. The proof never uses the sign of a weight, only that edges go forward in the order, and a DAG cannot contain a negative cycle. In a general graph a negative cycle makes distances -infinity, Dijkstra's finalized vertices can still be improved by a later negative edge, and a single pass in arbitrary order is not enough (Bellman-Ford needs V-1 passes).

5. Which algorithm when

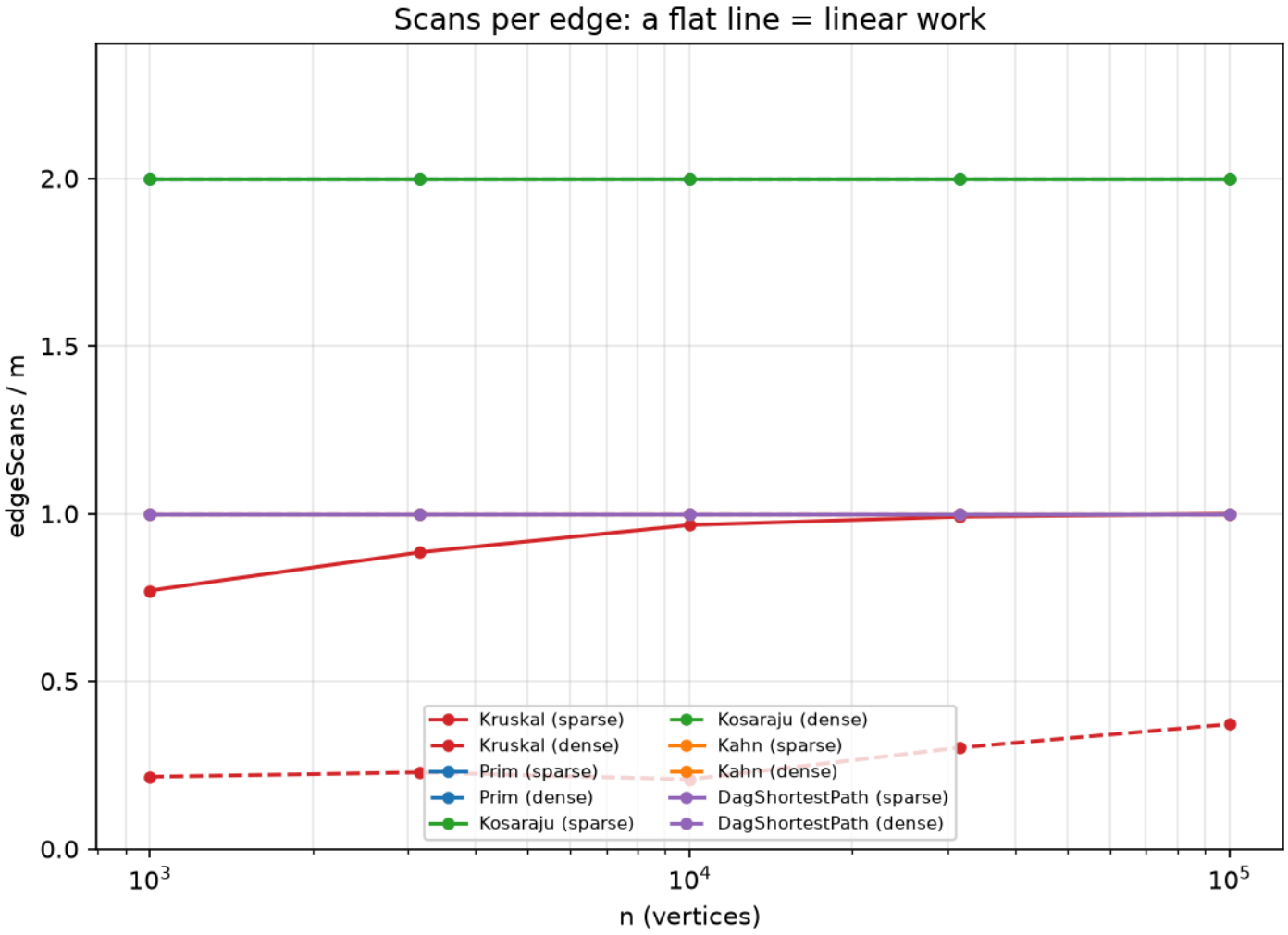
Situation	Choice	Why
MST, sparse graph, edge list	Kruskal	Input is the edge list; sort of E ~ 4V edges is cheap, DSU makes the cycle test almost O(1). At n = 100 000: 5.5M comparisons vs 14.4M for Prim. Gives a forest and component count for free.
MST, dense graph, adjacency list/matrix or implicit edges	Prim (array O(V^2) for m ~ V^2)	No global sort, O(V) memory for implicit edges. With a lazy binary heap our counters still favour Kruskal (2.6-2.8x fewer comparisons), so Prim wins mainly with the array version or implicit graphs.
MST, graph may be disconnected	Kruskal	Forest is the natural result; Prim needs a restart loop.
SCC of a directed graph	Kosaraju	O(V+E); needs the reversed graph. Explicit stack is required: measured maxDepth reaches 91 920 at n = 100 000.
Dependent jobs, cycle possible	Kahn	Fewer than V vertices leaving the queue is the cycle report, so a wrong order is never returned.
Shortest paths, acyclic, negative weights allowed	DAG relaxation	O(V+E) in one pass; Dijkstra needs non-negative weights, Bellman-Ford costs O(VE).
Shortest paths, cycles possible	Dijkstra (w >= 0) / Bellman-Ford	No topological order exists with cycles (not part of this assignment).

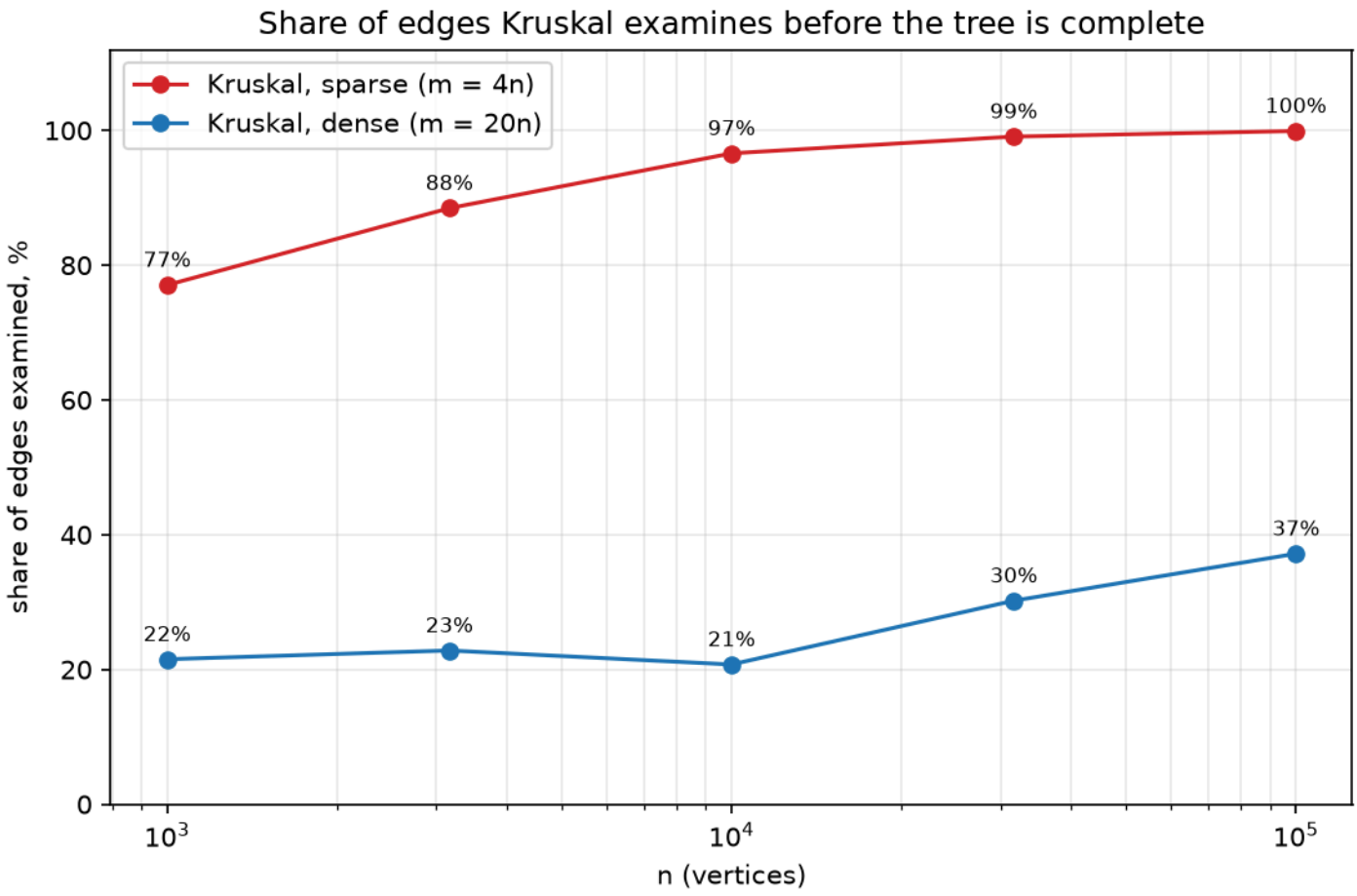
6. Plots

Plot1 - Operations vs n (log-log; Kosaraju and DagShortestPath both count 2m operations, their lines coincide).



Plot 2 - Scans per edge (edgeScans / m): flat = linear work. Prim and Kosaraju sit at 2, Kahn and DAG pass at 1; Kruskal rises with the share of examined edges. Plot 3 - Share of edges Kruskal examines: sparse 77 % (n = 1 000) to 99.9 % (n = 100 000), dense only 21-37 %. The early stop pays off only on dense graphs, and only for scans; the sort still orders all E edges.


Scans per edge: a flat line = linear work
edgeScans / m
2.0
1.5
1.0
0.5
0.0
10^3
10^4
10^5
n (vertices)
Kruskal (sparse)
Kruskal (dense)
Prim (sparse)
Prim (dense)
Kosaraju (sparse)
Kosaraju (dense)
Kahn (sparse)
Kahn (dense)
DagShortestPath (sparse)
DagShortestPath (dense)


Share of edges Kruskal examines before the tree is complete
share of edges examined, %
100
80
60
40
20
0
10^3
10^4
10^5
n (vertices)
Kruskal, sparse (m = 4n)
Kruskal, dense (m = 20n)
77%
88%
97%
99%
100%
22%
23%
21%
30%
37%

7. Time

Median of 5 runs (JDK 21, one shared virtual CPU). At small sizes the clock is noise: Kruskal on sparse graphs takes 74 ms at n = 3 162 but 43 ms at n = 10 000, Kahn on dense graphs 33 ms at n = 1 000 but 15 ms at n = 10 000 (JIT warm-up, GC, background load). On the largest dense graph the order Kahn 207 ms, DAG 209 ms, Kruskal 626 ms, Kosaraju 1 456 ms, Prim 2 552 ms agrees with the counters, but another machine or JVM would give other times. The counters (79.1M comparisons for Prim vs 27.9M for Kruskal there) are identical on every machine, so conclusions are drawn from counters, not from the clock.